Computer Architecture, Organization, and Performance

Lecture Notes — Week 1

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 8, 2026

1.1 Introduction

This chapter opens the course by asking what a computer is and, more precisely, how fast one is. A program is written in a high-level language, compiled to machine instructions, and run on hardware; the hardware that fetches, decodes, and executes those instructions is the subject of **computer organization**, while the instruction set and programmer-visible interface constitute **computer architecture**. Together they decide *how fast* a program runs and *what* it can do. Throughout this chapter we compare two machines running the *same* program: we count instructions, measure clock rate and CPI, compute execution times, and then ask how much a proposed hardware upgrade actually helps. The thread is quantitative throughout — speed is a number, not an adjective.

1.2 Architecture vs. Organization, and Basic Structure

1.2.1 The Interface and the Implementation

The distinction between architecture and organization is the first conceptual tool of the course. **Computer architecture** is what the programmer and compiler see: the instruction set, the registers, the data types, the addressing modes. Two machines that share an architecture can run the same binary — every modern x86 model is the same architecture. **Computer organization** is how the hardware realizes that interface: the datapath, the control unit, the bus structure, the memory hierarchy. Two machines with the same architecture can differ enormously in organization, and therefore in speed and cost.

Definition (Computer Architecture). *The programmer-visible interface of a machine — its instruction set, registers, data types, and addressing modes. Two machines with the same architecture run the same programs.*

Definition (Computer Organization). *The hardware implementation of that interface — the datapath, control unit, buses, and memory hierarchy. Same architecture, different organization, different speed.*

The distinction matters for two reasons. First, **compatibility**: keeping the architecture stable means old software keeps running on new hardware — the reason the x86 architecture has survived for decades. Second, **design freedom**: within one architecture, engineers may freely redesign the organization (pipelining, caches, faster ALUs) to make the same instructions execute faster. Patterson & Hennessy frame it as the "what" versus the "how": architecture is the programmer's contract, organization is the hardware's construction (P&H, Sec. 1.1, p.3) [2].

1.2.2 Classes of Computers

Computers come in classes with different design goals (Hamacher, Ch. 1; P&H, Sec. 1.1, p.3) [1, 2]. **Personal computers** serve one user with general-purpose workloads, balancing performance and cost. **Server computers** handle many users and large workloads, where availability and throughput matter most. **Supercomputers** are the fastest machines for the largest scientific problems, optimized for peak floating-point performance. **Embedded computers** live inside appliances, cars, and phones, where cost and power dominate and performance is just enough. **Personal mobile devices** (smartphones, tablets) are today the most common computers, constrained by power and size. The lesson: what is "best" for a phone is not what is "best" for a supercomputer; the design goal differs by class.

1.2.3 The Basic Structure of a Computer

The classic three-part structure (Figure 1.1) is a **CPU** (control unit + ALU + registers), **memory**, and **I/O** devices, connected by buses. The CPU is where computation happens; memory holds instructions and data; I/O connects the machine to the outside world (P&H, Sec. 1.4, "Under the Covers," p.16) [2].

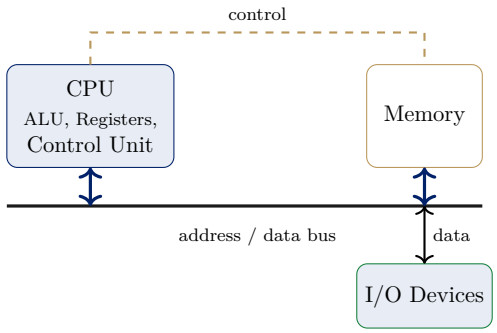

Figure 1.1: The basic structure of a computer — cf. P&H, *Computer Organization and Design*, ARM Edition, Sec. 1.4, p.16: CPU (ALU + registers + control) connected to memory and I/O by buses [2].

1.2.4 What Actually Runs: The Instruction Cycle Preview

The machine's heartbeat is the **instruction cycle**: the CPU repeatedly **fetches** an instruction from memory, **decodes** it, and **executes** it. Week 4 dissects this cycle in detail; here we need only its consequence. Execution time is built from three quantities: how many instructions run (IC, the instruction count), how many cycles each takes on average (CPI, cycles per instruction), and how fast a cycle is (the clock rate f, or equivalently the clock cycle time $T_c = 1/f$). Those three quantities are the entire subject of the performance equation, developed in the next section.

1.3 The Basic Performance Equation

1.3.1 Motivation: What Does "Faster" Mean?

Two machines run the same program. One has a faster clock, the other executes fewer instructions per program. Which is "faster" — and how do you decide without guessing? The only honest

measure is **time**: the execution time of the actual program. Everything else (clock rate, CPI, instruction count) is a *component* of time — useful for design, not a standalone claim of speed. The performance equation puts the three components together.

Definition (Basic Performance Equation).

$$\textit{CPU time} = \textit{instruction count} \times \textit{CPI} \times \textit{clock cycle time} = IC \times CPI \times T_c = \frac{IC \times CPI}{f},$$

where IC is the number of instructions executed, CPI is the average cycles per instruction, T_c is the clock cycle time, and $f = 1/T_c$ is the clock rate.

The three factors are set by different parts of the design (Table 1). **IC** is the instruction count — how many instructions the program executes, set by the program, compiler, and ISA (the architecture). **CPI** is the average cycles per instruction — set by the organization (datapath, pipeline, memory). f is the clock rate — set by fabrication technology and circuit design. Improving one factor does not automatically improve the others, and sometimes improving one hurts another: a more complex instruction can reduce IC but raise CPI.

Factor	What it measures	Influenced by	Changed by
IC	instructions per program	ISA, compiler, program	architecture
CPI	cycles per instruction	datapath, pipeline, memory	organization
f (T_c)	clock rate	fabrication, circuit design	technology + design

Table 1: The three factors of the performance equation and what sets each one.

1.3.2 Worked Examples: Two Machines, One Program

The running thread of the chapter is a quantitative comparison: program P executes 10^7 instructions, and we compare machines under different assumptions.

Example 1.3.1 (Two machines, one program — a tie)**.** *Machine A runs at $f = 2$ GHz with $CPI = 2$; Machine B runs at $f = 3$ GHz with $CPI = 3$.*

- `A:` $IC = 10^7$, $CPI = 2$, $T_c = 0.5$ ns $\Rightarrow$ $time = 10^7 \times 2 \times 0.5ns = 10$ ms.
- `B:` $IC = 10^7$, $CPI = 3$, $T_c = \frac{1}{3}$ ns $\Rightarrow$ $time = 10^7 \times 3 \times \frac{1}{3}ns = 10$ ms.

A tie, despite B's faster clock: B's higher CPI cancels the clock advantage. A faster clock does not imply a faster machine — the equation decides.

Example 1.3.2 (Breaking the tie by improving CPI)**.** *Machine B's designer reduces CPI from 3 to 1.5 (better pipelining). Recompute:*

$$\texttt{B:}\ CPI = 1.5,\ T_c = \tfrac{1}{3}ns\ \Rightarrow\ time = 10^7 \times 1.5 \times \tfrac{1}{3}ns = 5ms.$$

*Now B is 2× **faster** — the improvement came from CPI (organization), not clock rate. This is why the course spends Weeks 5–7 on datapath and control design: they directly set CPI.*

1.3.3 Socratic Check: Which Component Should You Improve?

A program spends its time entirely on arithmetic. You can either double the clock rate, or halve the CPI of arithmetic instructions. Assuming the rest stays fixed, both improve the same term (time $\propto$ CPI/f) by 2× — a tie for this program. But real programs mix instruction types: improving only the CPI of *one* type helps only the fraction of time that type occupies. That leads directly to Amdahl's Law (Section 1.5).

1.4 Measuring and Comparing Performance

1.4.1 What to Measure

The right measure is **execution time** of a real, representative workload — seconds, or its inverse, **throughput** (work done per second). **Benchmarks** are standard programs used to compare machines; the more representative of real workloads, the more believable the comparison. Two time definitions matter. **Wall-clock time** counts everything (memory, I/O, OS) — best for user-visible speed. **CPU time** counts only the time the CPU is computing on the program — best for isolating processor-design differences.

1.4.2 MIPS: A Metric to Be Careful With

MIPS (Million Instructions Per Second) is defined as

$$\text{MIPS} = \frac{\text{instruction count}}{\text{execution time} \times 10^6} = \frac{f}{\text{CPI} \times 10^6}.$$

It is misleading for three reasons (P&H, Sec. 1.6, p.28) [2]. It ignores what each instruction *does*: a machine with a harder instruction set can do more useful work per MIPS. It varies with the program: a program with more easy instructions gets a higher MIPS number. And two machines can report the same MIPS yet have very different execution times for the same program. The rule of thumb: report **time** for comparisons; use MIPS only with the caveat that it ignores the instruction set.

Example 1.4.1 (The MIPS trap when instruction sets match)**.** *Machine A: 1 GHz, CPI 2.0, runs program P with IC* $= 10^6$*. Machine B: 1 GHz, CPI 1.0, runs the same P.*

- `A:` *MIPS* $= \frac{10^9}{2\times10^6} = 500$*, time* $= \frac{10^6\times2}{10^9} = 2$ *ms.*
- `B:` *MIPS* $= \frac{10^9}{1\times10^6} = 1000$*, time* $= \frac{10^6\times1}{10^9} = 1$ *ms.*

Here B's MIPS is 2× and time is 2× better — consistent, because the instruction sets match. The trap appears only when the instruction sets differ; then MIPS numbers become incomparable, and time is always safe.

1.4.3 Socratic Check: Why Not Just Count Instructions?

Machine C executes 500 million instructions of program P in 1 second; Machine D executes 800 million instructions of the same P in 1.6 seconds. Computing time: C is 1 s, D is 1.6 s — *C is faster* despite running fewer instructions. Instruction count alone is meaningless without time: **IC** is a component of the equation, not a speed metric on its own.

1.5 Amdahl's Law

1.5.1 Motivation: Speeding Up One Part

A program takes 100 s. You can make the multiply part 10× faster, but multiplies are only 20% of the time. The whole program cannot be 10× faster: the unimproved 80% still takes its full time, so the new total is $80 + 2 = 82$ s, a speedup of roughly 1.22× — far from 10×. The principle generalizes.

Definition (Amdahl's Law). *If a fraction f of execution time can be sped up by a factor S_f, the overall speedup is*

$$\textit{Speedup} = \frac{1}{(1-f) + \dfrac{f}{S_f}}.$$

The unimproved fraction $(1-f)$ runs at full speed forever; only the improved fraction f/S_f shrinks (P&H, Sec. 1.6, p.28) [2].

The two consequences are the law's real content. As $S_f \to \infty$, the speedup approaches $1/(1-f)$ — a **hard ceiling** set by the unimproved fraction (Figure 1.2). Improving a part that is only a small fraction of the time buys almost nothing, no matter how fast you make it.

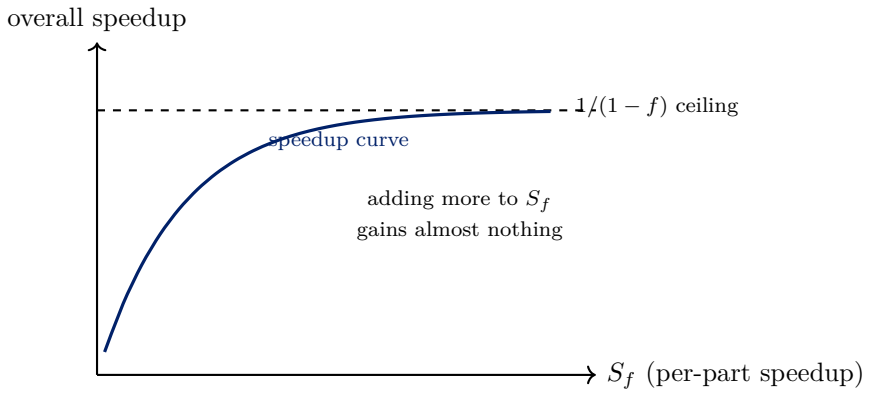

Figure 1.2: Amdahl's Law: the overall speedup saturates at $1/(1-f)$; an infinite speedup of the improved part cannot beat the unimproved fraction.

1.5.2 Worked Examples

Example 1.5.1 (Applying Amdahl's Law)**.** *40% of a program's time is floating-point; a new FP unit makes that part 4× faster.*

$$\textit{Speedup} = \frac{1}{(1-0.4) + 0.4/4} = \frac{1}{0.6 + 0.1} = \frac{1}{0.7} \approx 1.43\times.$$

Even a 4× FP speedup on a 40% fraction buys only 1.43× overall — the 60% unimproved part dominates.

Example 1.5.2 (The ceiling in action)**.** *The same program, 40% FP. The best possible overall speedup (FP part infinitely fast) is*

$$\lim_{S_f \to \infty} \frac{1}{(1-f) + f/S_f} = \frac{1}{1-f} = \frac{1}{0.6} \approx 1.67\times.$$

Design implication: if the goal is a 3× overall speedup and FP is only 40%, the FP unit is the wrong place to invest — *even infinite FP speedup cannot reach 3×. The other 60% must also be sped up.*

1.5.3 Socratic Check: The Parallelism Twist

Amdahl's Law is often quoted against parallel computing: "even with infinite processors, the serial fraction limits speedup." The serial fraction $(1-f)$ does cap speedup at $1/(1-f)$, so the statement is right for fixed-size workloads. But it carries a hidden assumption: that f is **fixed**. If the problem *grows* (larger input size), the parallel fraction usually grows with it, so larger problems extract more parallelism and soften the ceiling. Amdahl's Law is a statement about fixed-size workloads, not about scaling problems.

1.6 Synthesis and Summary

1.6.1 The Thread, Closed

The chapter's running comparison resolved a tie and a win. Machine A (2 GHz, CPI 2) and Machine B (3 GHz, CPI 3) both took 10 ms on a 10^7-instruction program; improving B's CPI to 1.5 made it 2× faster — a CPI (organization) win, not a clock-rate win. The comparison used every tool in the chapter: the performance equation to compute time, the factor table to see which lever was pulled, and time (not MIPS) as the comparison metric.

1.6.2 Bridge to Week 2

The performance equation assumes we can count instructions and CPI. But before the CPU can execute anything, the **data** it works on must be represented in bits, and arithmetic must be implemented in hardware. Week 2 covers **number systems**: how integers are represented (two's complement) and how adders and subtractors implement arithmetic in hardware — the "what the bits mean" layer that the instruction set operates on.

1.6.3 Summary

Architecture is the programmer's interface; **organization** is the hardware implementation; same architecture, different organization, different speed. The **basic structure** is CPU (ALU + registers + control) connected to memory and I/O by buses. The **performance equation** is CPU time = IC × CPI/f, with three independent factors. **Measurement** is time (or throughput) on a representative benchmark; beware MIPS. **Amdahl's Law** is Speedup = $1/[(1-f)+f/S_f]$ — the unimproved fraction sets an unbreakable ceiling.

References

[1] V. Carl Hamacher, Zvonko G. Vranesic, and Safwat G. Zaky. *Computer Organization*. McGraw-Hill, 5th edition, 2002.

[2] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, arm edition (5th) edition, 2017.